گزارش پروژه طراحی دیتابیس بانک

فاز اول:

به طور معمول ، در گام اول طراحی یک پایگاه داده برای یک سیستم ، باید ابتدا تمامی موجودیت های آن سیستم را درنظر بگیریم. همچنین ویژگی های هر موجودیت و ارتباط آنها با یکدیگر را هم مشخص میکنیم .

سپس جهت دید بصری به موجودیت های تعریف شده ، نمودار ER را برای آن رسم میکنیم.

پس ابتدا به تمامی موجودیت های ممکن یک دیتابیس بانک فکر میکنیم.
موجودیت ها می‌توانند بصورت زیر باشند.

Entites (موجودیت ها)

1. **شعبه(Branch) :**

- کد شعبه (PK)

- نام شعبه
- شهر
- تلفن
- آدرس

2. **مشتری(Customer) :**

- کد مشتری /کد ملی (PK)
- نام
- نام خانوادگی
- تاریخ تولد
- شماره تلفن همراه
- آدرس سکونت

3. **کارمند(Employee) :**

- کد پرسنلی (PK)
- نام
- نام خانوادگی
- سمت (رئیس، تحویل‌دار، پشتیبان و...)
- حقوق
- تاریخ استخدام

4. **حساب(Account) :**

- شماره حساب (PK)

- نوع حساب (جاری، پس‌انداز، سپرده بلندمدت)
- موجودی
- تاریخ افتتاح
- وضعیت حساب (فعال، مسدود، راکد)

5. **کارت بانکی(Card) :**

- شماره کارت (PK- ۱۶ رقمی)
- رمز دوم
- CVV2
- تاریخ انقضا
- سقف برداشت روزانه

6. **تراکنش(Transaction) :**

- کد تراکنش (PK)
- نوع تراکنش (واریز، برداشت، کارت‌به‌کارت، پایا)
- مبلغ
- تاریخ و ساعت
- وضعیت تراکنش (موفق، ناموفق)

7. **وام(Loan) :**

- کد وام (PK)
- نوع تسهیلات (مسکن، ازدواج، خودرو)
- مبلغ کل وام

- نرخ سود (درصد)
- تعداد اقساط
- وضعیت وام (در جریان، تسویه شده)

8. **قسط(Installment) :**

- شناسه قسط (PK)
- شماره قسط (مثلاً قسط اول، قسط دوم...)
- مبلغ قسط
- تاریخ سررسید
- تاریخ پرداخت
- وضعیت پرداخت (پرداخت شده، معوقه)

9. **ضامن(Guarantor) :**

- کد ملی ضامن (PK)
- نام
- نام خانوادگی
- شماره تلفن
- نوع ضمانت (فیش حقوقی، چک، سفته)

10. **چک(Cheque) :**

- شماره صیادی (PK)
- مبلغ چک
- تاریخ سررسید

- نام گیرنده
- وضعیت چک (پاس شده، برگشتی، در جریان)

روابط بین موجودیت‌ها، کاردینالیتی و مشارکت

حال به این میپردازیم که موجودیت هایی که تعریف کردیم ، چه ارتباطاتی با یکدیگر دارند؟!

1. **رابطه شعبه و کارمند (استخدام):**

- هر شعبه چندین کارمند دارد. هر کارمند حتماً در یک شعبه کار می‌کند.
- **کاردینالیتی :**۱ به چند$(1:N)$
- **مشارکت :**از سمت شعبه اختیاری (شاید شعبه جدید کارمند نداشته باشد)، از سمت کارمند اجباری (کارمند بدون شعبه نداریم).

2. **رابطه شعبه و حساب (افتتاح حساب):**

- در هر شعبه چندین حساب افتتاح می‌شود. هر حساب متعلق به یک شعبه مشخص است .
- **کاردینالیتی :**۱ به چند$(1:N)$
- **مشارکت :**برای هر دو اجباری.

3. **رابطه مشتری و حساب (مالکیت حساب):**

- یک مشتری می‌تواند چندین حساب داشته باشد. یک حساب می‌تواند متعلق به چندین مشتری باشد (حساب‌های مشترک) .

- **کاردینالیتی :**چند به چند$(M:N)$
- **مشارکت :**برای مشتری اختیاری (می‌تواند فقط اطلاعاتش ثبت شده باشد ولی هنوز حساب باز نکرده باشد)، برای حساب اجباری (حساب بدون صاحب نداریم).

4. **رابطه حساب و کارت (صدور کارت):**

- برای یک حساب می‌توان چندین کارت (مثلاً المثنی یا برای شرکا) صادر کرد. هر کارت فقط به یک حساب وصل است.
- **کاردینالیتی :**۱ به چند$(1:N)$
- **مشارکت :**برای حساب اختیاری (حساب بدون کارت داریم)، برای کارت اجباری.

5. **رابطه حساب و تراکنش (انجام تراکنش):**

- یک حساب می‌تواند چندین تراکنش (مبدا یا مقصد) داشته باشد. هر تراکنش مربوط به یک حساب (یا دو حساب در جابجایی‌ها) است .
- **کاردینالیتی :**۱ به چند$(1:N)$
- **مشارکت :**برای حساب اختیاری، برای تراکنش اجباری

6. **رابطه مشتری و وام (دریافت وام):**

- یک مشتری می‌تواند چندین وام بگیرد. هر وام دقیقاً به یک مشتری پرداخت می‌شود .
- **کاردینالیتی :**۱ به چند$(1:N)$

- **مشارکت :**برای مشتری اختیاری، برای وام اجباری.

7. **رابطه وام و قسط (بخش‌بندی وام):**

- هر وام شامل چندین قسط است. هر قسط دقیقاً متعلق به **یک** وام است .
- **کاردینالیتی :**۱ به چند$(1:N)$
- **مشارکت :**برای هر دو اجباری.

8. **رابطه وام و ضامن (ضمانت وام):**

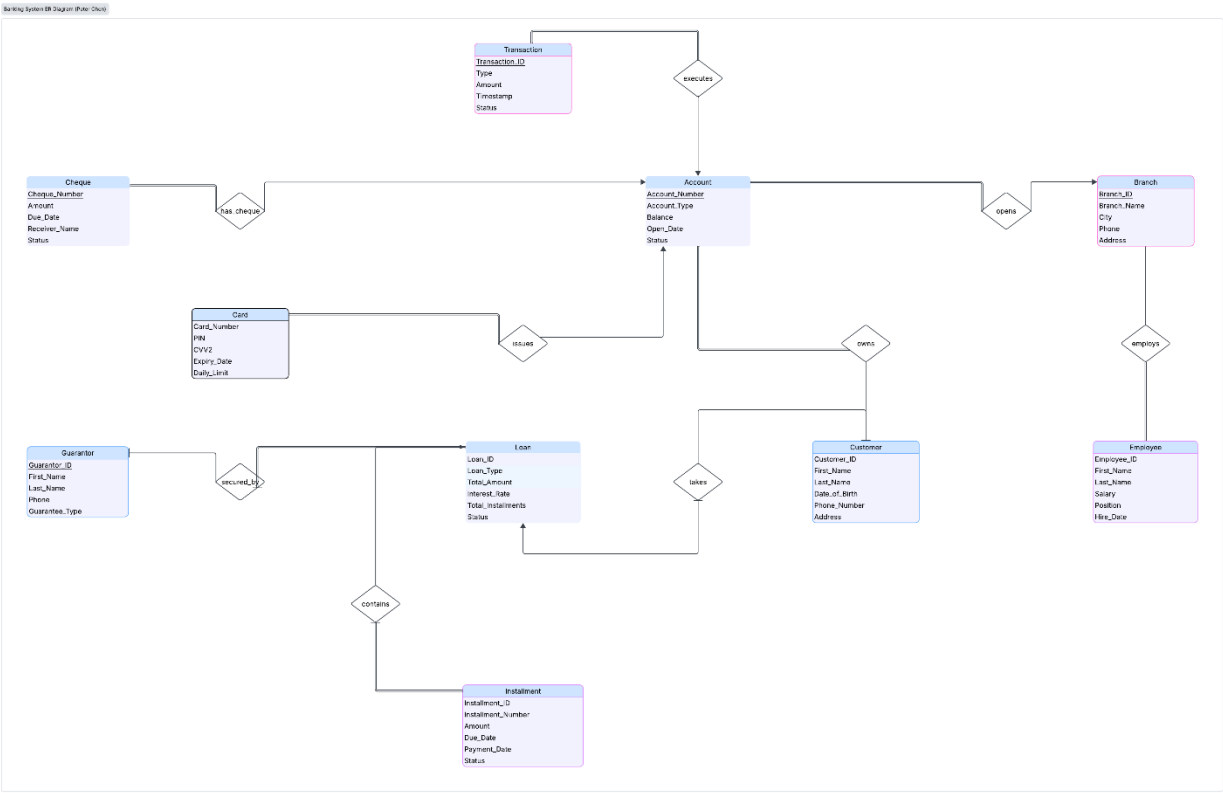
- یک وام می‌تواند توسط چندین ضامن ضمانت شود. یک ضامن می‌تواند ضامنِ چندین وام مختلف شود.
- **کاردینالیتی :**چند به چند$(M:N)$
- **مشارکت :**برای وام اجباری (وام بدون ضامن نمی‌دهیم)، برای ضامن اختیاری.

9. **رابطه حساب و چک (دسته چک):**

- از یک حساب جاری می‌توان چندین برگه چک صادر کرد. هر برگه چک متعلق به یک حساب جاری مشخص است.
- **کاردینالیتی :**۱ به چند$(1:N)$
- **مشارکت :**برای حساب اختیاری (همه حساب‌ها جاری نیستند)، برای چک اجباری.

نمودار E-R دیتابیس

نمودار ER دیتابیس طراحی شده را میتوانید مشاهده کنید (چون در این گزارش کیفیت فایل نامناسب است ، عکس واضع و دقیق با قابلیت زوم کردن در فایلهای پروژه با نام E-R diagram.png موجود است)



فاز دوم:

چون در نمودار فاز اول، تمام ویژگی‌های موجودیت‌ها ساده و تک‌مقداری بودند، جدول‌های ما خودبه‌خود در سطح **3NF**(فرم نرمال سوم) قرار دارند و نیازی به شکستن بیشتر ندارند؛ چرا که هیچ وابستگی انتقالی یا جزئی در آن‌ها وجود ندارد.

پس جداول رابطه ای عبارتند از :

۱.جدول شعبه(branch)

- Primary Key: branch_id
- Foreign Key : ندارد.
- Unique Constraint: branch_name (نام شعبه نباید تکراری باشد.)
- Check Constraint: ندارد.
- Default Constraint: ندارد.
- Not Null Constraint: branch_name, city

۲ .جدول مشتری(customer)

- Primary Key: customer_id
- Foreign Key: ندارد.
- Unique Constraint: phone_number
- Check Constraint: ندارد.
- Default Constraint: ندارد.
- Not Null Constraint:first_name, last_name

۳ .جدول کارمند(employee)

- **Primary Key**:employee_id
- **Foreign Key**: branch_id به branch(branch_id) (رابطه :employs چون سمت کارمند دوخطی بود، FKکارمند نمی‌تواند تهی باشد) .
- **Unique Constraint**: ندارد.
- **Check Constraint**:salary > 0
- **Default Constraint**: hire_date مقدار پیش‌فرض تاریخ روز(CURRENT_DATE)
- **Not Null Constraint**:first_name, last_name, position, salary, branch_id

۴ .جدول حساب(account)

- **Primary Key**:account_number
- **Foreign Key**: branch_id به branch(branch_id) رابطه(opens)
- **Unique Constraint**: ندارد.
- **Check Constraint**: balance >= 0 و

  status IN ('Active', 'Blocked', 'Closed')
- **Default Constraint**: balance مقدار پیش‌فرض 0؛ statusمقدار پیش‌فرض 'Active'؛ open_date مقدار پیش‌فرض تاریخ روز.
- **Not Null Constraint**:account_type, balance, open_date, status, branch_id

۵ .جدول کارت(card)

- **Primary Key**:card_number
- **Foreign Key**: account_number به account(account_number) رابطه (issuses)
- **Unique Constraint**: ندارد.
- **Check Constraint**:daily_limit > 0

- **Default Constraint**: daily_limit مقدار پیش‌فرض 50000000 (مثلاً ۵۰ میلیون ریال)
- **Not Null Constraint**:pin, cvv2, expiry_date, daily_limit, account_number

۶ .جدول تراکنش(transaction)

- **Primary Key**:transaction_id
- **Foreign Key**: account_number به account(account_number) رابطه (executes.)
- **Unique Constraint**: ندارد.
- **Check Constraint**: amount > 0 و

  status IN ('Success', 'Failed', 'Pending')
- **Default Constraint**: timestamp مقدار پیش‌فرض زمان جاری سیستم (CURRENT_TIMESTAMP)؛ statusمقدار پیش‌فرض 'Success'.
- **Not Null Constraint**:type, amount, timestamp, status, account_number

۷ .جدول وام(loan)

- **Primary Key**:loan_id
- **Foreign Key**: customer_id به customer(customer_id) رابطه(takes)
- **Unique Constraint**: ندارد.
- **Check Constraint**: total_amount > 0 و interest_rate >= 0 وtotal_installments > 0
- **Default Constraint**: status مقدار پیش‌فرض 'Active'.
- **Not Null Constraint**: loan_type, total_amount, interest_rate, total_installments, status, customer_id

۸ .جدول قسط(installment)

- **Primary Key**:installment_id

- **Foreign Key**: loan_id به loan(loan_id) رابطه (contains)
- **Unique Constraint**: ندارد.
- **Check Constraint**: amount > 0 و

  status IN ('Paid', 'Unpaid', 'Overdue')
- **Default Constraint**: status مقدار پیش‌فرض 'Unpaid'.
- **Not Null Constraint**:installment_number, amount, due_date, status, loan_id

۹ .جدول چک(cheque)

- **Primary Key**:cheque_number
- **Foreign Key**: account_number به account(account_number) رابطه .(has_cheques)
- **Unique Constraint**: ندارد.
- **Check Constraint**:amount > 0
- **Default Constraint**: status مقدار پیش‌فرض 'Pending'.
- **Not Null Constraint**:amount, due_date, receiver_name, status, account_number

۱۰ .جدول واسط صاحبان حساب — (account_owners) رابطه چند به چندowns

- **Primary Key**: ترکیبی از .(customer_id, account_number)
- **Foreign Key1**: customer_idبه .customer(customer_id)
- **Foreign Key2**: account_number به account(account_number)
- **Unique Constraint**: ندارد.
- **Check Constraint**: ندارد.
- **Default Constraint**: ندارد.

- customer_id, account_number:**Not Null Constraint**

۱۱ .**جدول واسط ضامنین وام — (loan_guarantors)** رابطه چند به چندsecured_by

- **Primary Key**: ترکیبی از (loan_id, guarantor_id)
- **Foreign Key1**: loan_id به loan(loan_id)
- **Foreign Key 2**: guarantor_id به guarantor(guarantor_id)
- **Unique Constraint**: ندارد.
- **Check Constraint**: ندارد.
- **Default Constraint**: ندارد.
- loan_id, guarantor_id:**Not Null Constraint**

۱۲ .**جدول ضامن(guarantor)**

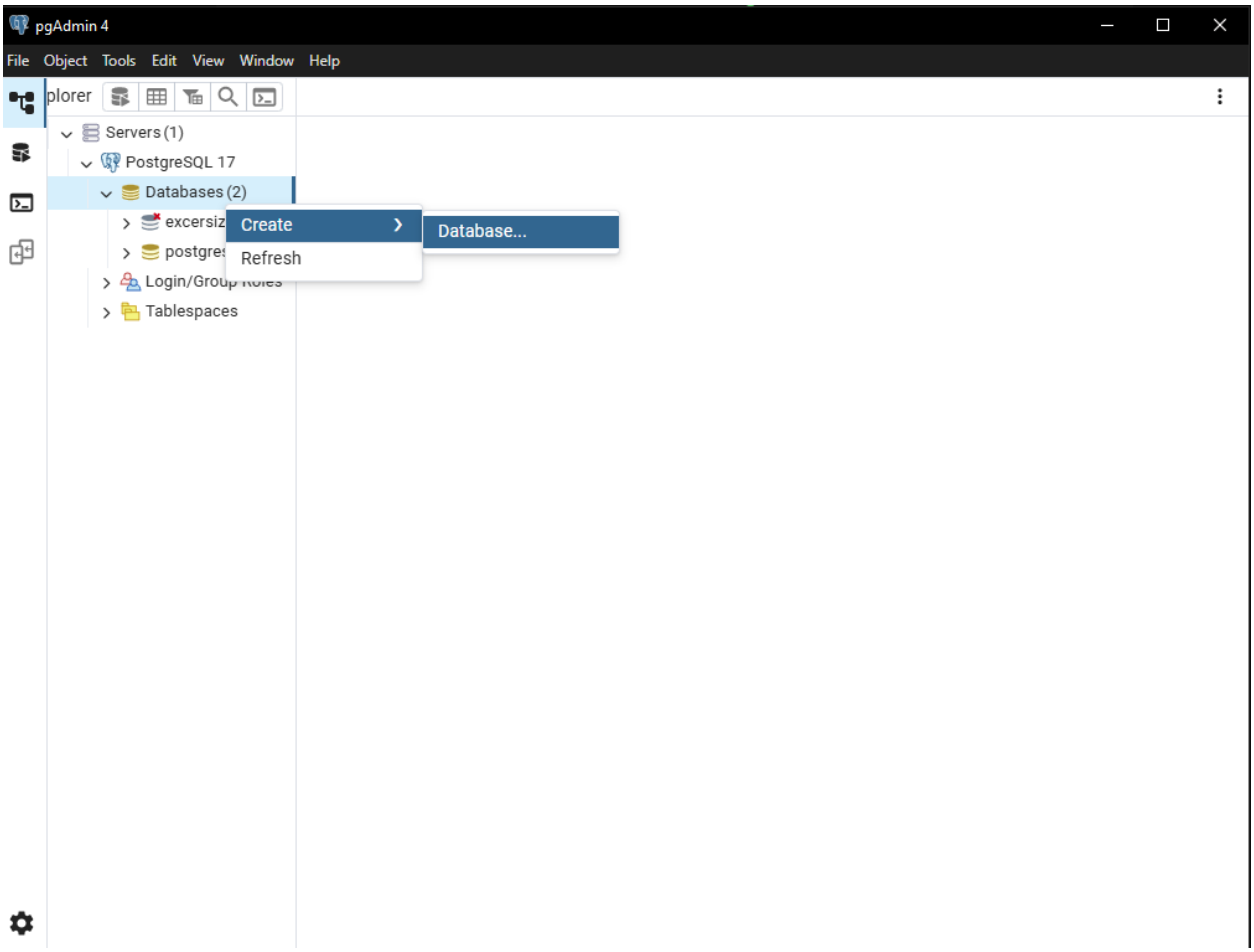
- guarantor_id:**Primary Key**
- **Foreign Key**: ندارد.
- phone:**Unique Constraint**
- **Check Constraint**: ندارد.
- **Default Constraint**: ندارد.
- first_name, last_name, guarantee_type:**Not Null Constraint**

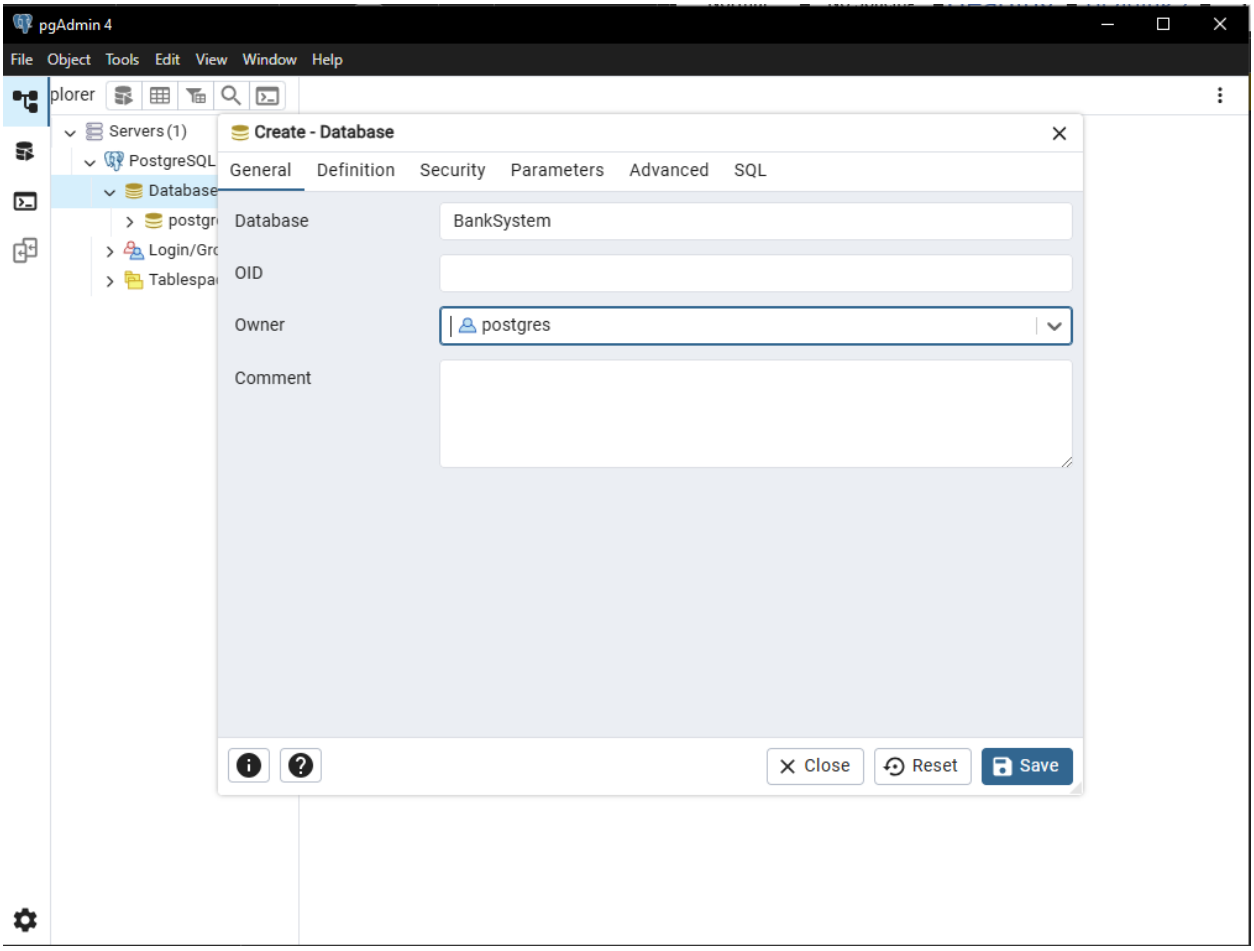
فاز سوم

در این فاز می‌بایست دیتابیسی که طراحی کرده ایم و جداول و روابطش معلوم هستند را در SQL Server پیاده سازی کنیم.(من از postgresql و سرور pgadmin4 استفاده میکنم و با query tool دستورات رو خواهیم نوشت)

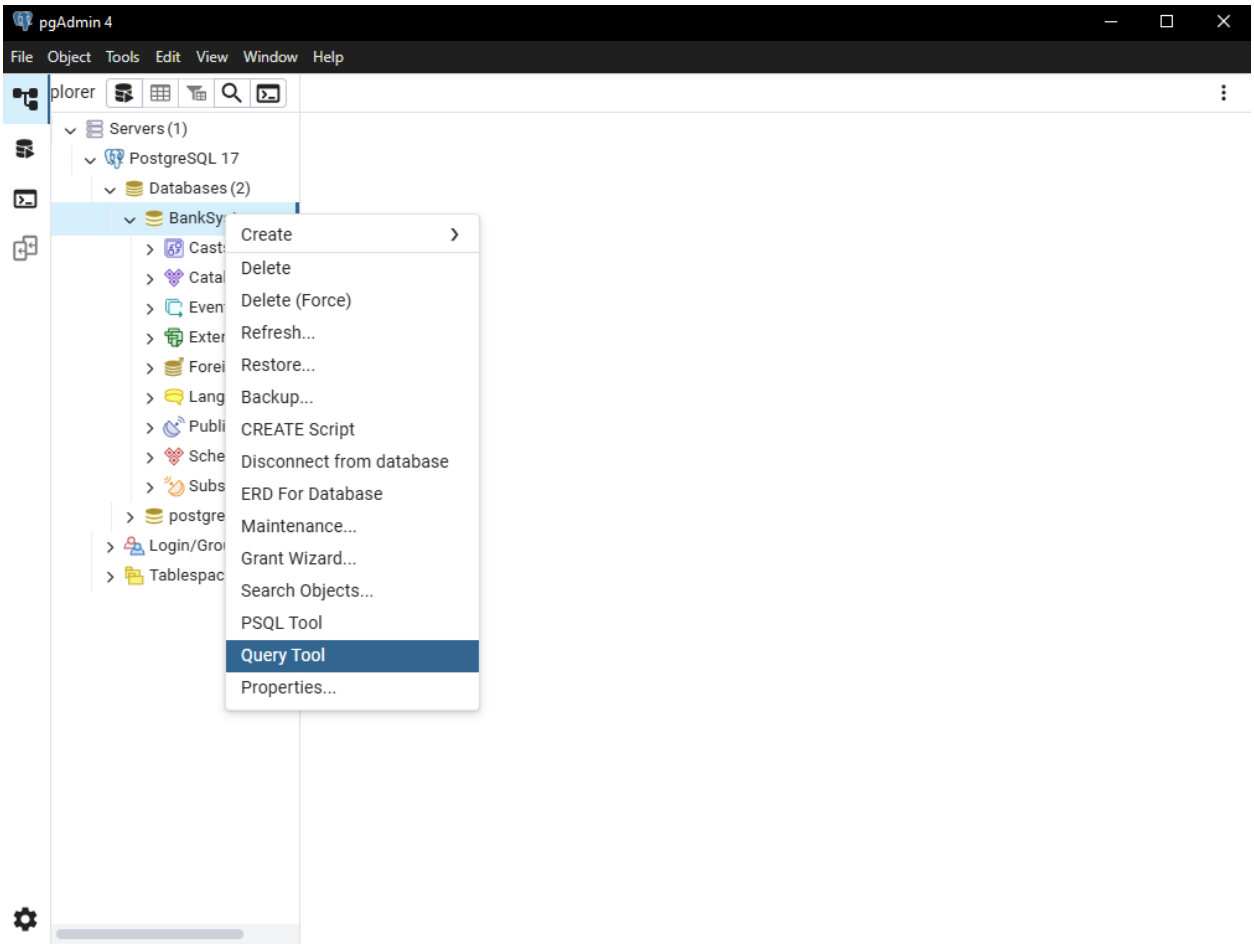
مراحل کار را بصورت گزارش وار نمایش میدهم:

1.ابتدا یک دیتابیس ایجاد میکنیم و نامش را BankSystem میگذاریم:

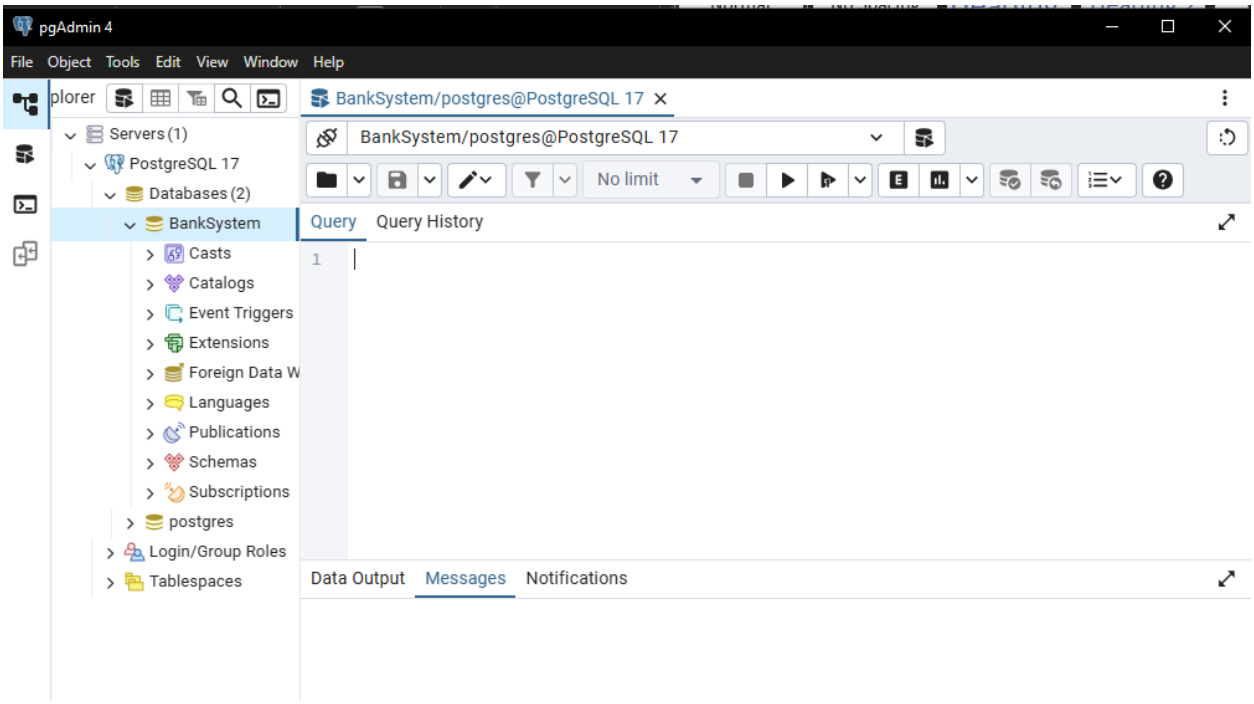




سپس از گزینه query tool استفاده میکنیم تا فضایی برای وارد کردن کوئری ها و کد های sql مان داشته باشیم:

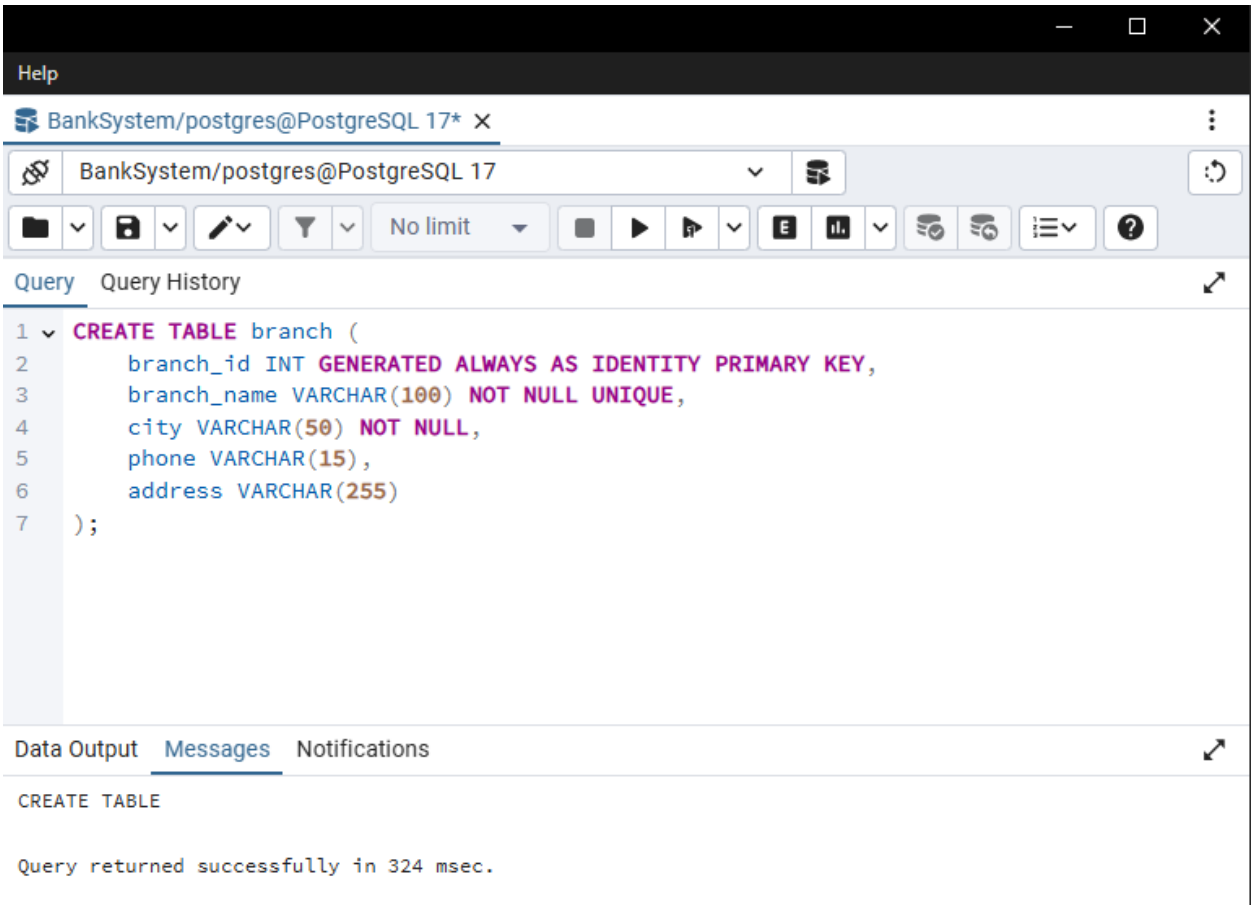

pgAdmin 4
File
Object
Tools
Edit
View
Window
Help
plorer
Servers (1)
PostgreSQL 17
Databases (2)
Create
Delete
Delete (Force)
Refresh...
Restore...
Backup...
CREATE Script
Disconnect from database
ERD For Database
Maintenance...
Grant Wizard...
Search Objects...
PSQL Tool
Query Tool
Properties...

در اینجا کدهای sql را وارد میکنیم :



حال جداول را یکی پس از دیگری با دستورات زیر ایجاد میکنیم (فایل کامل کوئری های ساخت دیتابیس در فایل create.sql در مسیر اصلی پروژه موجود است!):

تیبل branch یا شعبه :



و... به همین منوال تمامی تیبل های موردنیاز را میسازیم(تاکید میشود فایل کامل ساخت تیبل های دیتابیس درون create.sql است.)

نکته (تفاوت های سینتکسی بین sql server و postgres):

1. تولید خودکار کلیدهای اصلی(Auto-Increment) :

درSQL Server : از دستور IDENTITY(start, increment) استفاده می‌شود.

درPostgreSQL : از استاندارد SQL استفاده می‌شود که به صورت GENERATED ALWAYS AS IDENTITY نوشته می‌شود. همچنین برای مشخص کردن شروع شماره‌گذاری از یک عدد خاص (مثل شماره حساب‌ها از ۱۰۰۰)، ویژگی (START WITH 1000) درون پرانتز قرار می‌گیرد.

2. توابع زمان و تاریخ سیستم(System Date & Time) :

درSQL Server : از تابع GETDATE()برای دریافت تاریخ و زمان فعلی استفاده می‌شود.

درPostgreSQL : از کلیدواژه‌های استاندارد CURRENT_DATE(فقط برای تاریخ) و CURRENT_TIMESTAMP(برای تاریخ و زمان دقیق همراه با ثانیه) استفاده می‌شود.

3. کلمات کلیدی رزرو شده(Reserved Keywords) :

درSQL Server : کلمه Transactionیک دستور رزرو شده است، اما با قرار دادن آن داخل کروشه [Transaction]می‌توان جدول را ساخت.

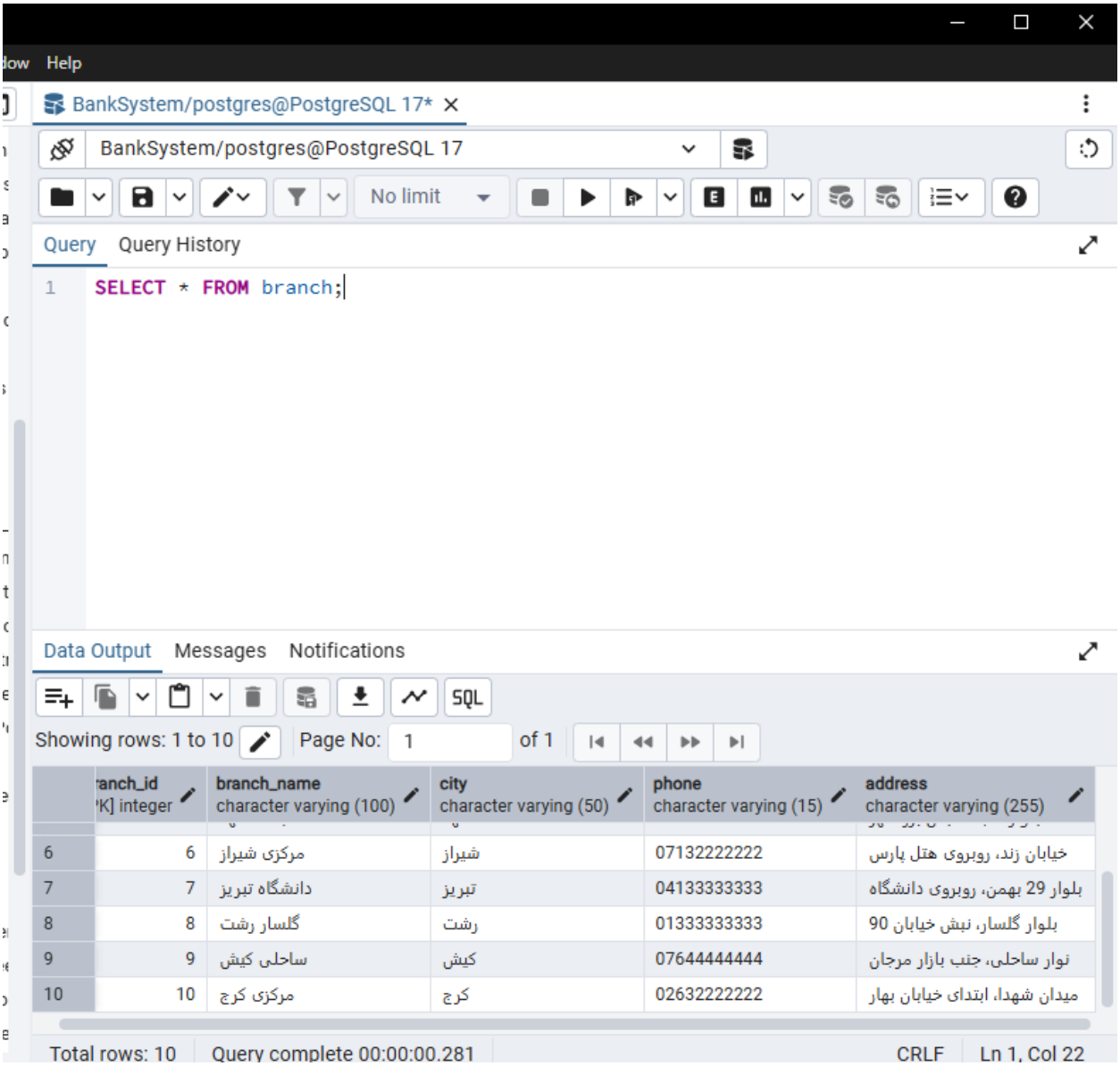
درPostgreSQL : استفاده از کلمات رزرو شده حتی با ساختارهای دیگر هم چالش‌برانگیز است؛ به همین دلیل برای سازگاری کامل و جلوگیری از خطاهای ناخواسته، نام جدول را به یک نام استانداردتر یعنی transaction_logتغییر دادیم.

3. insert کردن 10 داده در هر جدول :

کوئری مورد نظر در فایل insert.sql موجود است!

نکته : در سیستم پایگاه دادهPostgreSQL ، برای مقداردهی دستی به فیلدهایی که با ویژگی GENERATED ALWAYS AS IDENTITYتعریف شده‌اند، جهت حفظ یکپارچگی ارجاعی (Referential Integrity) و جلوگیری از خطای قید کلید خارجی(FK Violation) ، از عبارت استاندارد OVERRIDING SYSTEM VALUEاستفاده شد.

حال چک میکنیم که دیتاهای موردنظرمان در تیبل ها قرارگرفتند یا خیر:



میبینیم که عملیات با موفقیت انجام شده است.

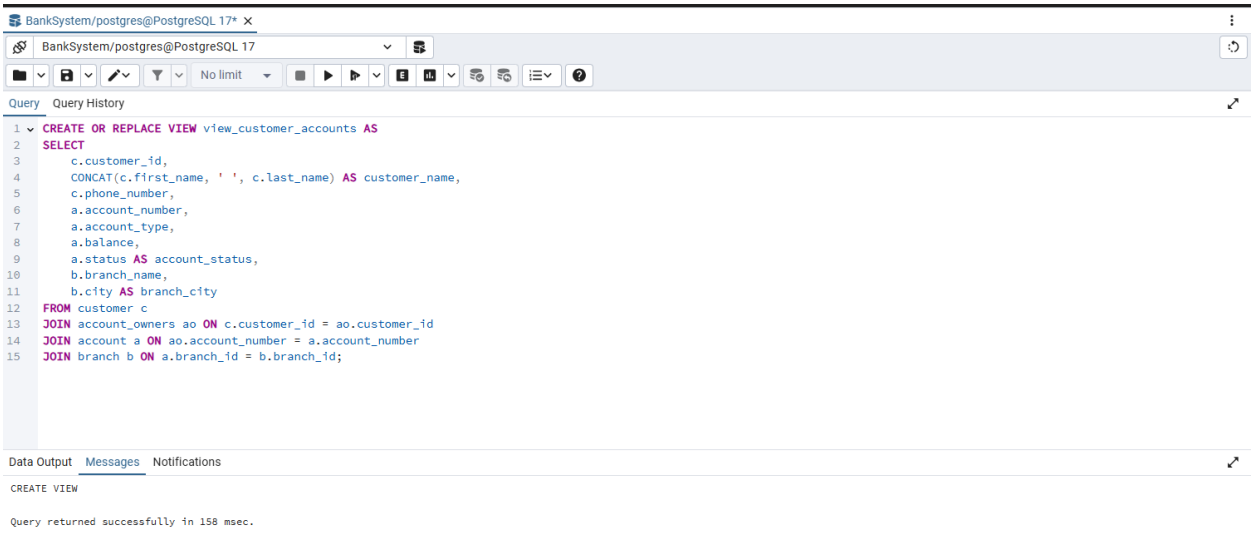
حال میتوانیم به تعریف view ها ، function ها ، تریگر و پرسیژر بپردازیم! در ادامه همچنان بصورت گزارش وار از روند کار مطلع خواهید شد...

View : اگر بخواهیم خیلی ساده بگوییم، Viewیک جدول مجازی (Virtual Table) است.

یک View خودش به تنهایی هیچ داده‌ای را درون دیتابیس یا هارد دیسک ذخیره نمی‌کند. در واقع، Viewچیزی نیست جز یک **کوئری SELECTذخیره شده** که یک نام مشخص دارد. وقتی شما یک View را صدا می‌زنید، دیتابیس پشت صحنه می‌رود، کوئریِ اصلی آن را اجرا می‌کند و خروجی را طوری به شما نشان می‌دهد که انگار دارید به یک جدول واقعی نگاه می‌کنید.

دو ویو طراحی میکنیم : (کوئری این ویو های تعریف شده در views.sql می‌باشد!)

ویوی اول :(view_customer_accounts) ترکیب اطلاعات مشتری، شماره حساب، نوع حساب، موجودی و نام شعبه‌ای که حساب در آن باز شده است. (این کار دسترسی به اطلاعات کلی حساب‌های مشتریان را به شدت ساده می‌کند).



Query Query History

```
SELECT * FROM view_customer_accounts;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	customer_id integer	customer_name text	phone_number character varying (15)	account_number integer	account_type character varying (50)	balance numeric (18,2)	account_status character varying (20)	branch_name character varying (100)	branch_city character varying (50)
1	1	علی رضایی	09121111111	1000	جاری	500000000.00	Active	مرکزی تهران	تهران
2	1	علی رضایی	09121111111	1001	پس‌انداز	12000000.00	Active	مرکزی تهران	تهران
3	2	زهرا احمدی	09122222222	1002	سپرده بلندمدت	2000000000.00	Active	آزادی	تهران
4	3	محمد کریمی	09131111111	1003	جاری	85000000.00	Active	دانشگاه خواجه نصیر	تهران
5	4	ساره محمدی	09151111111	1004	پس‌انداز	4500000.00	Active	دانشگاه خواجه نصیر	تهران
6	4	ساره محمدی	09151111111	1005	جاری	150000000.00	Active	مرکزی اصفهان	اصفهان
7	2	زهرا احمدی	09122222222	1005	جاری	150000000.00	Active	مرکزی اصفهان	اصفهان
8	5	حسین حسینی	09171111111	1006	پس‌انداز	30000000.00	Active	سجاد مشهد	مشهد
9	6	مریم قاسمی	09141111111	1007	سپرده بلندمدت	5000000000.00	Active	مرکزی شیراز	شیراز
10	7	رضا صادقی	09111111111	1009	جاری	9500000.00	Active	دانشگاه تبریز	تبریز

ویوی دوم :(view_overdue_installments) نمایش لیست تمام مشتریانی که اقساط معوقه و پرداخت‌نشده دارند، به همراه مبلغ قسط، شماره تلفن مشتری و نوع وام، برای استفاده بخش پیگیری مطالبات بانک.

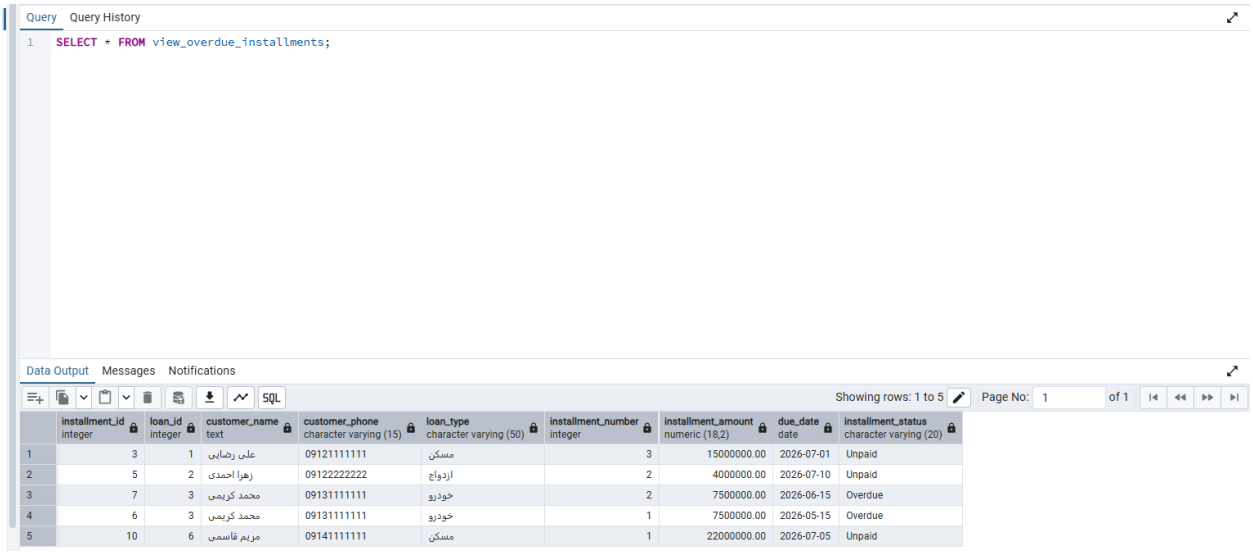
Query Query History

```
CREATE OR REPLACE VIEW view_overdue_installments AS
SELECT
    i.installment_id,
    l.loan_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    c.phone_number AS customer_phone,
    l.loan_type,
    i.installment_number,
    i.amount AS installment_amount,
    i.due_date,
    i.status AS installment_status
FROM installment i
JOIN loan l ON i.loan_id = l.loan_id
JOIN customer c ON l.customer_id = c.customer_id
WHERE i.status IN ('Overdue', 'Unpaid') AND i.due_date < CURRENT_DATE;
```

Data Output Messages Notifications

```
CREATE VIEW

Query returned successfully in 126 msec.
```



	installment_id integer	loan_id integer	customer_name text	customer_phone character varying (15)	loan_type character varying (50)	installment_number integer	installment_amount numeric (18,2)	due_date date	installment_status character varying (20)
1	3	1	علی رضایی	09121111111	مسکن	3	15000000.00	2026-07-01	Unpaid
2	5	2	زهرا احمدی	09122222222	ازدواج	2	4000000.00	2026-07-10	Unpaid
3	7	3	محمد کریمی	09131111111	خودرو	2	7500000.00	2026-06-15	Overdue
4	6	3	محمد کریمی	09131111111	خودرو	1	7500000.00	2026-05-15	Overdue
5	10	6	مریم قاسمی	09141111111	مسکن	1	22000000.00	2026-07-05	Unpaid

Function ها: اگر ویو را یک «جدول مجازی یا پنجره‌ای رو به داده‌ها» در نظر بگیریم، **فانکشن مثل یک کارخانه یا ماشین پردازش کوچک است.**

فانکشن کدی است که می‌تواند ورودی (ورودی‌ها یا پارامترها) بگیرد، یک‌سری محاسبات، پردازش‌ها یا عملیات منطقی را روی آن‌ها انجام دهد و در نهایت یک خروجی مشخص را برگرداند.(RETURN)

تفاوت:

ویو(View) : پارامتر ورودی نمی‌گیرد. فقط یک کوئریِ ثابت است که کل داده‌های متصل را نشان می‌دهد.

فانکشن(Function) : پویا است. به آن ورودی می‌دهید (مثلاً شماره حساب یک مشتری خاص) و او دقیقاً پردازش را برای همان ورودی انجام می‌دهد.

نکته: کوئری های دو فانکشن تعریف شده ، در فایل functions.sql موجود است.

1.پیاده‌سازی تابع(fn_get_total_successful_transactions) Scalar : این تابع شماره حساب را می‌گیرد و مجموع تراکنش‌های موفقش را به صورت یک عدد برمی‌گرداند.

Query Query History

```sql
CREATE OR REPLACE FUNCTION fn_get_total_successful_transactions(p_account_number INT)
RETURNS DECIMAL(18,2) AS $$
DECLARE
    v_total DECIMAL(18,2);
BEGIN
    SELECT COALESCE(SUM(amount), 0)
    INTO v_total
    FROM transaction_log
    WHERE account_number = p_account_number AND status = 'Success';

    RETURN v_total;
END;
$$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

```
CREATE FUNCTION

Query returned successfully in 97 msec.
```

تست کردن تابع:

Query Query History

```sql
SELECT fn_get_total_successful_transactions(1000) AS total_success_amount;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	total_success_amount numeric
1	15000000.00

پیاده‌سازی تابع Table-valued (fn_get_branch_employees) :

این تابع شناسه یک شعبه را می‌گیرد و لیست کارمندان آن شعبه را به صورت جدول خروجی می‌دهد.

Query Query History

```sql
CREATE OR REPLACE FUNCTION fn_get_branch_employees(p_branch_id INT)
RETURNS TABLE (
    employee_id INT,
    full_name TEXT,
    emp_position VARCHAR(50),
    salary DECIMAL(18,2),
    hire_date DATE
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        e.employee_id,
        CONCAT(e.first_name, ' ', e.last_name)::TEXT,
        e.position,
        e.salary,
        e.hire_date
    FROM employee e
    WHERE e.branch_id = p_branch_id;
END;
$$ LANGUAGE plpgsql;
```

Data Output Messages Notifications

```
CREATE FUNCTION

Query returned successfully in 128 msec.
```

تست خروجی تابع دوم:

Query Query History

```sql
SELECT * FROM fn_get_branch_employees(1);
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	employee_id integer	full_name text	emp_position character varying	salary numeric	hire_date date
1	11	احمد مرادی	رئیس شعبه	450000000.00	2015-03-10
2	12	…وفر کاظمی	معاون شعبه	350000000.00	2018-06-15
3	13	سعید اسدی	تحویل‌دار	200000000.00	2020-09-01

Stored procedure : پروسیجر مجموعه‌ای از دستورات SQL ذخیره‌شده است که برای انجام یک عملیات یا یک «بیزینس لاجیک» خاص نوشته می‌شود.

تفاوتش با فانکشن:

هدف فانکشن :معمولاً برای انجام **محاسبات** و برگشت دادن یک مقدار یا جدول استفاده می‌شود (RETURNدارد) .فانکشن‌ها را نمی‌توان برای تراکنش‌های سنگین بانکی که نیاز به تایید نهایی یا لغو دارند استفاده کرد .

هدف پروسیجر :برای انجام **عملیات و رفتارهای سیستمی** است. پروسیجر نیازی به ریترن کردن مقدار ندارد. مهم‌ترین ویژگی پروسیجر این است که می‌تواند فرآیند کنترل تراکنش یعنی COMMIT(ثبت قطعی در دیتابیس) و ROLLBACK(لغو کل عملیات در صورت بروز خطا) را مدیریت کند.

مثال بانکی ما (بند ۱۳) :عملیات **انتقال وجه** بین دو حساب .این عملیات شامل کم کردن پول از حساب اول، اضافه کردن به حساب دوم و ثبت یک لاگ تراکنش است. اگر وسط کار برق قطع شود یا حساب دوم مسدود باشد، پروسیجر کل عملیات را ROLLBACKمی‌کند تا پول غیب نشود!

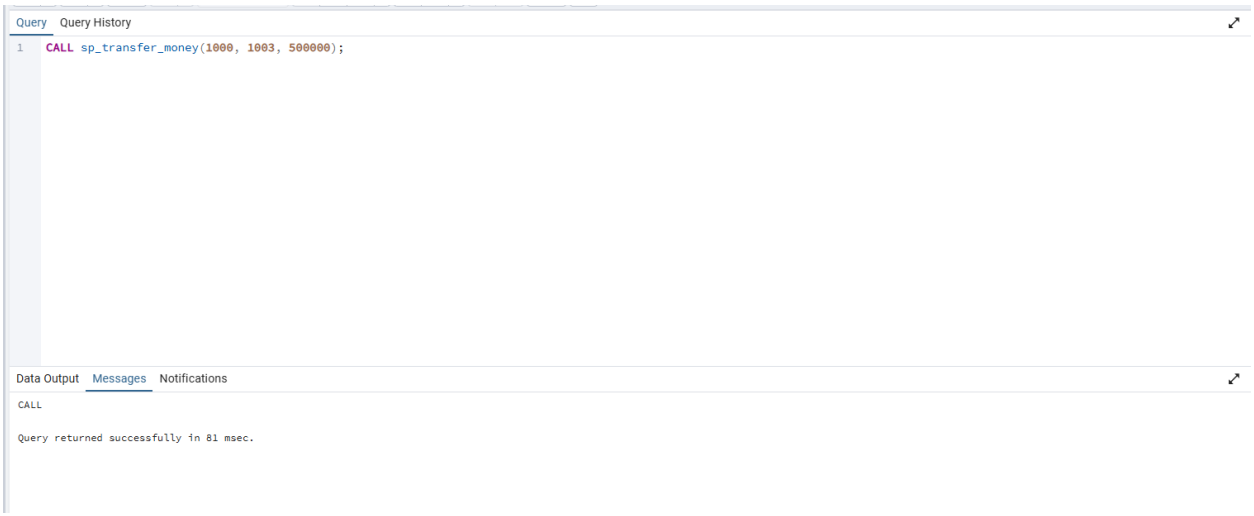
نکته: کوئری های پرسیجر و تریگر تعریف شده ، درون trigger_and_procedure.sql موجود است.

پیاده‌سازی Stored Procedure برای انتقال وجه(sp_transfer_money) :

این پروسیجر شماره حساب مبدا، مقصد و مبلغ را می‌گیرد. ابتدا موجودی مبدا را چک می‌کند، اگر کافی بود پول را جابه‌جا کرده و تراکنش‌ها را ثبت می‌کند.



تست کردن پرسیجر:



تریگر (trigger):

تریگر یعنی «ماشه». تریگر یک رویداد خودکار است که ما آن را اجرا نمی‌کنیم، بلکه خودش در پاسخ به یک اتفاق (مثل INSERT، UPDATEیا DELETE) روی یک جدول، بیدار می‌شود و شلیک می‌کند!

به زبان ساده، شما به دیتابیس می‌گویید هر زمان که کسی خواست روی جدول تراکنش‌ها یک رکورد جدید بگذارد(INSERT) ، بلافاصله قبل یا بعد از آن، برو و موجودی جدول حساب‌ها را به‌روزرسانی کن.

- **ویژگی اصلی :**تریگرها برای کاربر یا برنامه‌نویس کاملاً مخفی هستند و در پشت صحنه امنیت و یکپارچگی داده‌ها را تضمین می‌کنند.
- **مثال بانکی ما (بند ۱۴) :**تریگر خودکار برای به‌روزرسانی موجودی حساب .وقتی یک تراکنش «واریز» موفق ثبت می‌شود، تریگر خود به خود موجودی آن حساب را افزایش می‌دهد و نیازی نیست ما دستی این کار را بکنیم.

پیاده سازی تریگر برای بروزرسانی خودکار موجودی حساب:

در PostgreSQL ساخت تریگر دو مرحله دارد: اول ساخت یک تابع مخصوص تریگر (FUNCTION) و دوم متصل کردن آن به جدول.(TRIGGER)

1. ساخت تریگر:

```
CREATE OR REPLACE FUNCTION trg_update_account_balance_func()
RETURNS TRIGGER AS $$
BEGIN
    -- اگر تراکنش موفق بود موجودی را تغییر بده
    IF NEW.status = 'Success' THEN
        IF NEW.type = 'واریز' OR NEW.type LIKE 'واریز%' THEN
            UPDATE account
            SET balance = balance + NEW.amount
            WHERE account_number = NEW.account_number;
        ELSIF NEW.type = 'برداشت' OR NEW.type LIKE 'برداشت%' THEN
            UPDATE account
            SET balance = balance - NEW.amount
            WHERE account_number = NEW.account_number;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 60 msec.
```

2. متصل کردن تریگر به جدول تراکنش ها:

```
CREATE TRIGGER trg_after_transaction_insert
AFTER INSERT ON transaction_log
FOR EACH ROW
EXECUTE FUNCTION trg_update_account_balance_func();
```

Data Output | Messages | Notifications

```
CREATE TRIGGER

Query returned successfully in 60 msec.
```

3. **دستور تست تریگر:** کافیست یک تراکنش جدید ثبت کنیم، تریگر خودش موجودی حساب ۱۰۰۱ را کسر می‌کند:

```
INSERT INTO transaction_log (type, amount, status, account_number)
VALUES ('1000000 ,برداشت', 'Success', 1001);

SELECT account_number, balance FROM account WHERE account_number = 1001;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	account_number [PK] integer	balance numeric (18,2)
1	1001	11000000.00

فاز چهارم

در این بخش باید کوئری های خواسته شده را بزنیم و از خروجی شان اسکرین شات بگذاریم:

نکته: کوئری ها همگی در فایل queries.sql موجود هستند.

کوئری اول: نمایش اطلاعات تمام مشتریان:

Query Query History

```
1  SELECT * FROM customer;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	customer_id [PK] integer	first_name character varying (50)	last_name character varying (50)	date_of_birth date	phone_number character varying (15)	address character varying (255)
1	1	علی	رضایی	1990-05-12	09121111111	...ران، سعادت آباد، خیابان ۲۴
2	2	زهرا	احمدی	1995-08-22	09122222222	...ان، میرداماد، شمس تبریزی
3	3	محمد	کریمی	1988-12-05	09131111111	اصفهان، خیابان خاقانی
4	4	ساره	محمدی	1992-03-14	09151111111	مشهد، بلوار هاشمیه
5	5	حسین	حسینی	2000-01-01	09171111111	شیراز، معالی آباد
6	6	مریم	قاسمی	1993-07-19	09141111111	تبریز، ولیعصر
7	7	رضا	صادقی	1985-10-30	09111111111	رشت، منظریه
8	8	مهدی	نوری	1997-02-25	09351111111	کرج، جهانشهر
9	9	فاطمه	امیدی	1991-09-11	09362222222	تهران، نارمک، میدان هفتاد
10	10	امیر	تیموری	1989-04-08	09123333333	تهران، یوسف آباد

کوئری دوم: نمایش مشتریانی که بیش از یک حساب دارند:

Query Query History

```
1  SELECT
2      c.customer_id,
3      CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
4      COUNT(ao.account_number) AS account_count
5  FROM customer c
6  JOIN account_owners ao ON c.customer_id = ao.customer_id
7  GROUP BY c.customer_id, c.first_name, c.last_name
8  HAVING COUNT(ao.account_number) > 1;
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	customer_id [PK] integer	customer_name text	account_count bigint
1	4	ساره محمدی	2
2	2	زهرا احمدی	2
3	1	علی رضایی	2

کوئری سوم: حساب هایی که بیش از 10000 موجودی دارند:

```
SELECT *
FROM account
WHERE balance > 10000;
```

Data Output | Messages | Notifications

Showing rows: 1 to 9 | Page No: 1 of 1

	account_number [PK] integer	account_type character varying (50)	balance numeric (18,2)	open_date date	status character varying (20)	branch_id integer
1	1002	سپرده بلندمدت	2000000000.00	2019-05-10	Active	2
2	1004	پس‌انداز	4500000.00	2023-02-11	Active	3
3	1005	جاری	150000000.00	2020-08-25	Active	4
4	1006	پس‌انداز	30000000.00	2021-12-05	Active	5
5	1007	سپرده بلندمدت	5000000000.00	2018-11-20	Active	6
6	1009	جاری	9500000.00	2022-09-15	Active	7
7	1000	جاری	499500000.00	2020-01-15	Active	1
8	1003	جاری	85500000.00	2022-06-01	Active	3
9	1001	پس‌انداز	11000000.00	2021-03-20	Active	1

کوئری چهارم: نمایش تعداد حساب های هر شعبه:

```
SELECT
    b.branch_id,
    b.branch_name,
    COUNT(a.account_number) AS total_accounts
FROM branch b
LEFT JOIN account a ON b.branch_id = a.branch_id
GROUP BY b.branch_id, b.branch_name;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No: 1 of 1

	branch_id [PK] integer	branch_name character varying (100)	total_accounts bigint
1	4	مرکزی اصفهان	1
2	10	مرکزی کرج	0
3	6	مرکزی شیراز	1
4	2	آزادی	1
5	9	ساحلی کیش	0
6	7	دانشگاه تبریز	1
7	3	دانشگاه خواجه نصیر	2
8	1	مرکزی تهران	3
9	5	سجاد مشهد	1
10	8	گلسار رشت	0

کوئری های join (کوئری پنجم): نمایش نام مشتری و شماره حساب او:

Query Query History

```
SELECT
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    ao.account_number
FROM customer c
JOIN account_owners ao ON c.customer_id = ao.customer_id;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	customer_name text	account_number integer
1	علی رضایی	1000
2	علی رضایی	1001
3	زهرا احمدی	1002
4	محمد کریمی	1003
5	ساره محمدی	1004
6	ساره محمدی	1005
7	زهرا احمدی	1005
8	حسین حسینی	1006
9	مریم قاسمی	1007
10	رضا صادقی	1009

کوئری های join (کوئری ششم): نمایش تراکنش های هر مشتری :

Query Query History

```
SELECT
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    t.transaction_id,
    t.type AS transaction_type,
    t.amount,
    t.timestamp,
    t.status
FROM customer c
JOIN account_owners ao ON c.customer_id = ao.customer_id
JOIN transaction_log t ON ao.account_number = t.account_number
ORDER BY c.customer_id, t.timestamp DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

	customer_name text	transaction_id [PK] integer	transaction_type character varying (50)	amount numeric (18,2)	timestamp timestamp without time zone	status character varying (20)
1	علی رضایی	13	برداشت	1000000.00	2026-07-20 04:30:46.539501	Success
2	علی رضایی	11	برداشت (کارت به کارت)	500000.00	2026-07-20 04:23:13.000399	Success
3	علی رضایی	3	کارت‌به‌کارت	2000000.00	2026-07-03 09:00:00	Success
4	علی رضایی	2	برداشت	5000000.00	2026-07-02 14:15:00	Success
5	علی رضایی	1	واریز	10000000.00	2026-07-01 10:30:00	Success
6	زهرا احمدی	7	کارت‌به‌کارت	800000.00	2026-07-06 11:12:00	Success
7	محمد کریمی	12	واریز (کارت به کارت)	500000.00	2026-07-20 04:23:13.000399	Success
8	محمد کریمی	5	واریز	45000000.00	2026-07-05 18:20:00	Success
9	محمد کریمی	4	پایا	150000000.00	2026-07-04 07:45:00	Success
10	ساره محمدی	7	کارت‌به‌کارت	800000.00	2026-07-06 11:12:00	Success
11	ساره محمدی	6	برداشت	12000000.00	2026-07-05 19:00:00	Failed
12	حسین حسینی	8	واریز	30000000.00	2026-07-07 13:50:00	Success
13	مریم قاسمی	9	ساتنا	500000000.00	2026-07-08 12:00:00	Success
14	رضا صادقی	10	برداشت	2500000.00	2026-07-09 16:33:00	Success

کوئری های join (کوئری هفتم):نمایش مشتریانی که وام دارند:

```
SELECT DISTINCT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    c.phone_number
FROM customer c
JOIN loan l ON c.customer_id = l.customer_id;
```

Data Output | Messages | Notifications

Showing rows: 1 to 9 | Page No: 1 of 1

	customer_id [PK] integer	customer_name text	phone_number character varying (15)
1	6	مریم قاسمی	09141111111
2	5	حسین حسینی	09171111111
3	4	ساره محمدی	09151111111
4	1	علی رضایی	09121111111
5	7	رضا صادقی	09111111111
6	2	زهرا احمدی	09122222222
7	8	مهدی نوری	09351111111
8	3	محمد کریمی	09131111111
9	10	امیر تیموری	09123333333

کوئری های aggregate (کوئری هشتم):میانگین موجودی حساب ها در هر شعبه

```
SELECT
    b.branch_id,
    b.branch_name,
    ROUND(AVG(a.balance), 2) AS average_balance
FROM branch b
JOIN account a ON b.branch_id = a.branch_id
GROUP BY b.branch_id, b.branch_name;
```

Data Output | Messages | Notifications

Showing rows: 1 to 7 | Page No: 1 of 1

	branch_id [PK] integer	branch_name character varying (100)	average_balance numeric
1	4	مرکزی اصفهان	150000000.00
2	6	مرکزی شیراز	5000000000.00
3	2	آزادی	2000000000.00
4	7	دانشگاه تبریز	9500000.00
5	3	دانشگاه خواجه نصیر	45000000.00
6	1	مرکزی تهران	170166666.67
7	5	سجاد مشهد	30000000.00

کوئری های aggregate(کوئری نهم) : بیشترین تراکنش انجام شده در هر شعبه:

Query Query History

```sql
SELECT
    b.branch_id,
    b.branch_name,
    MAX(t.amount) AS max_transaction_amount
FROM branch b
JOIN account a ON b.branch_id = a.branch_id
JOIN transaction_log t ON a.account_number = t.account_number
GROUP BY b.branch_id, b.branch_name;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	branch_id [PK] integer	branch_name character varying (100)	max_transaction_amount numeric
1	4	مرکزی اصفهان	800000.00
2	6	مرکزی شیراز	500000000.00
3	7	دانشگاه تبریز	2500000.00
4	3	دانشگاه خواجه نصیر	150000000.00
5	1	مرکزی تهران	10000000.00
6	5	سجاد مشهد	30000000.00

کوئری های aggregate (کوئری دهم): تعداد وام های هر مشتری :

Query Query History

```sql
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    COUNT(l.loan_id) AS total_loans
FROM customer c
LEFT JOIN loan l ON c.customer_id = l.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	customer_id [PK] integer	customer_name text	total_loans bigint
1	4	ساره محمدی	1
2	10	امیر تیموری	1
3	6	مریم قاسمی	1
4	2	زهرا احمدی	1
5	9	فاطمه امیدی	0
6	7	رضا صادقی	1
7	3	محمد کریمی	1
8	1	علی رضایی	2
9	5	حسین حسینی	1
10	8	مهدی نوری	1

کوئری های nested (کوئری یازدهم):

Query Query History

```sql
SELECT DISTINCT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.balance
FROM customer c
JOIN account_owners ao ON c.customer_id = ao.customer_id
JOIN account a ON ao.account_number = a.account_number
WHERE a.balance > (SELECT AVG(balance) FROM account);
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	customer_id integer	customer_name text	balance numeric (18,2)
1	6	مریم قاسمی	5000000000.00
2	2	زهرا احمدی	2000000000.00

کوئری های nested (کوئری دوازدهم) نمایش شعبه ای که بیشترین تعداد حساب را دارد:

Query Query History

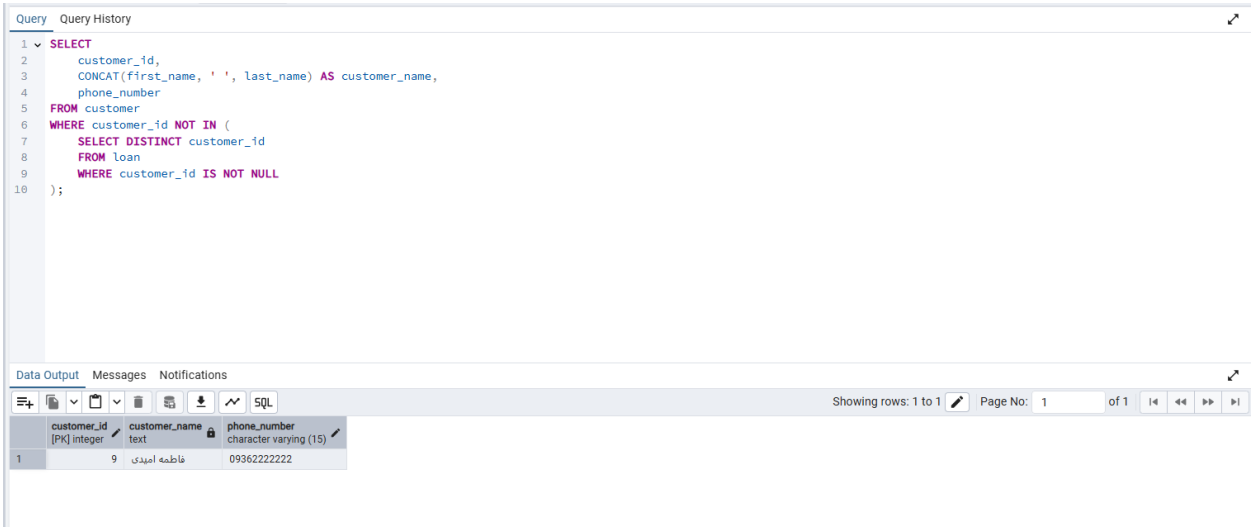
```sql
SELECT
    branch_id,
    branch_name,
    account_count
FROM (
    SELECT
        b.branch_id,
        b.branch_name,
        COUNT(a.account_number) AS account_count
    FROM branch b
    LEFT JOIN account a ON b.branch_id = a.branch_id
    GROUP BY b.branch_id, b.branch_name
) AS branch_counts
ORDER BY account_count DESC
LIMIT 1;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	branch_id [PK] integer	branch_name character varying (100)	account_count bigint
1	1	مرکزی تهران	3

کوئری های nested (کوئری سیزدهم و آخر) : نمایش مشتریانی که هیچ وامی ندارند:

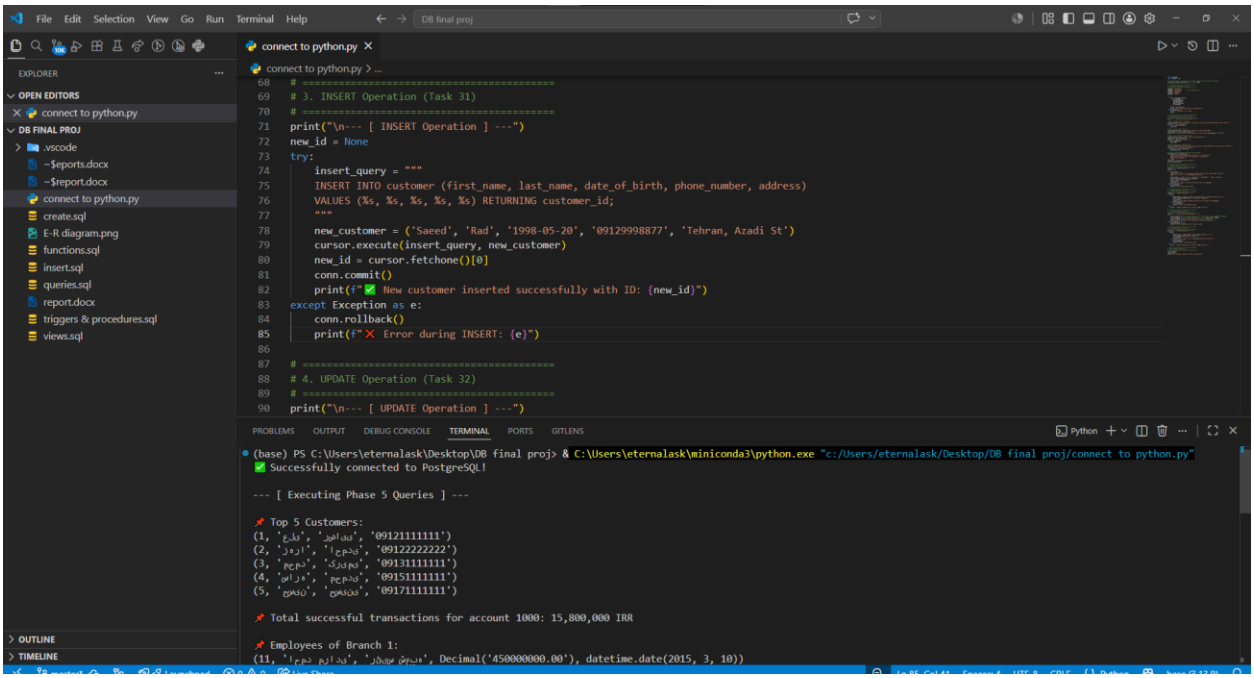


فاز پنجم و آخر

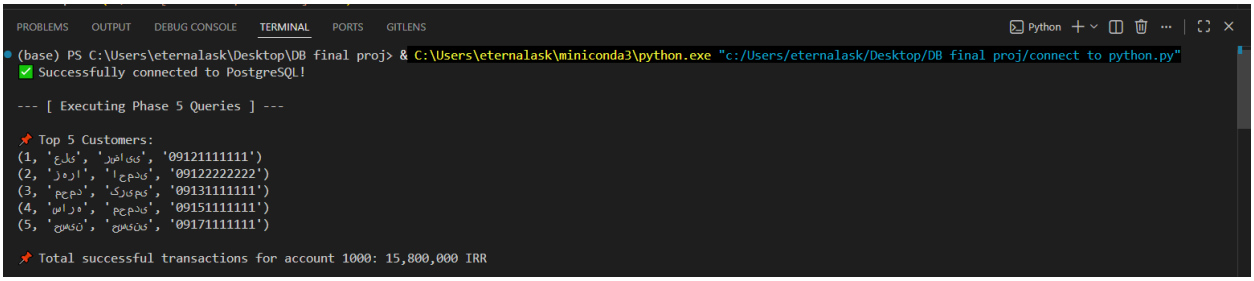
چون بجای sql server از postgresql استفاده میکنیم ، میبایست بجای کتابخانه pyodbc که برای اتصال پایتون به sql server است، از psycopg2 استفاده کنیم که مختص postgresql طراحی شده است. کتابخانه psycopg2محبوب‌ترین، پرکاربردترین و استانداردترین درایور (Adapter) زبان پایتون برای متصل شدن به دیتابیس PostgreSQL است!

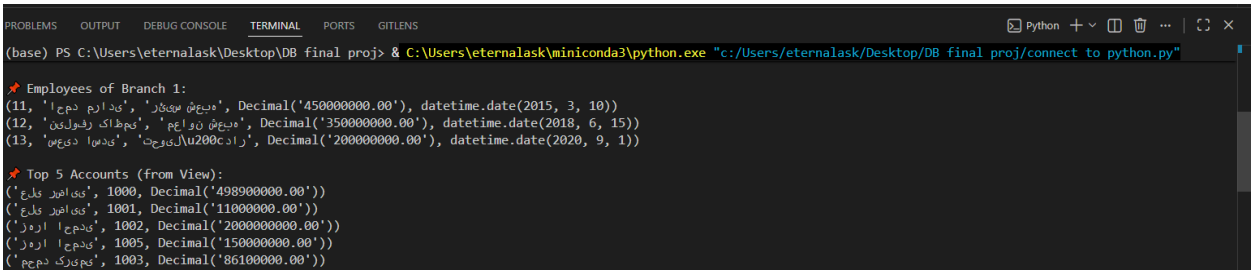
فاز پنجم و ارتباط گیری با دیتابیس از طریق کد پایتونی، در فایل connect to python.py موجود در فایلهای پروژه بطور کامل پیاده سازی شده است و میتوانید آنرا مشاهده کنید. اما در این گزارش ، اسکرین شاتی از ران شدن کد قرار میدهیم تا صحت آن تایید شود.

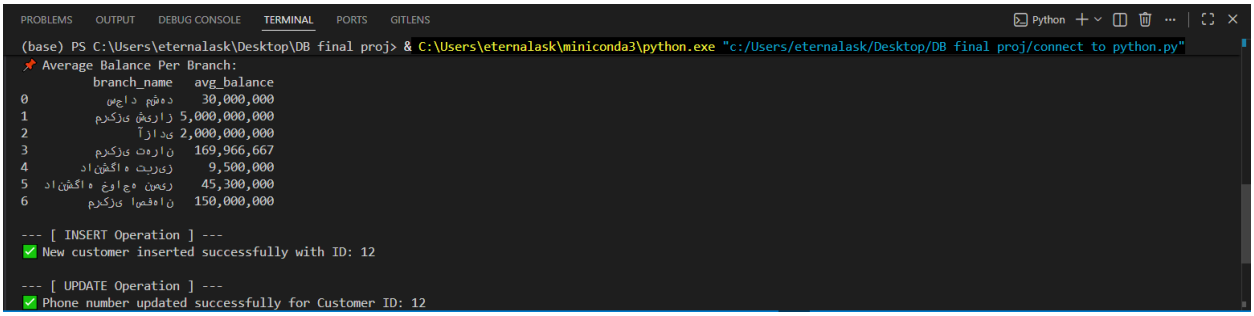
پس از ران شدن کد:

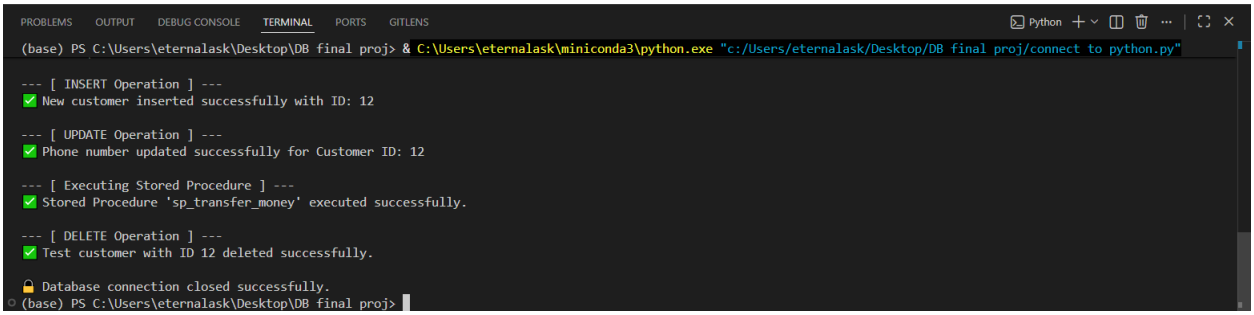


اسکرین شات های دقیقتر از خروجی:






PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS Python
(base) PS C:\Users\eternalask\Desktop\DB final proj> & C:\Users\eternalask\miniconda3\python.exe "c:/Users/eternalask/Desktop/DB final proj/connect to python.py"
📌 Average Balance Per Branch:
branch_name avg_balance
0 دهقم داعیس 30,000,000
1 زاریش نزکرم 5,000,000,000
2 یدازآ 2,000,000,000
3 نزکرم نارهت 169,966,667
4 زیربت هاگشن اد 9,500,000
5 ریبن هجاوخ هاگشن اد 45,300,000
6 نزکرم اهفاه ن 150,000,000
--- [INSERT Operation] ---
✅ New customer inserted successfully with ID: 12
--- [UPDATE Operation] ---
✅ Phone number updated successfully for Customer ID: 12


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS Python
(base) PS C:\Users\eternalask\Desktop\DB final proj> & C:\Users\eternalask\miniconda3\python.exe "c:/Users/eternalask/Desktop/DB final proj/connect to python.py"
--- [INSERT Operation] ---
✅ New customer inserted successfully with ID: 12
--- [UPDATE Operation] ---
✅ Phone number updated successfully for Customer ID: 12
--- [Executing Stored Procedure] ---
✅ Stored Procedure 'sp_transfer_money' executed successfully.
--- [DELETE Operation] ---
✅ Test customer with ID 12 deleted successfully.
🔒 Database connection closed successfully.
(base) PS C:\Users\eternalask\Desktop\DB final proj>